

DATA STRUCTURE (CS100)

UNIT II — Linear Data Structures

LINKED LIST

BTechX

□ Syllabus — Unit II

- Single Linked List: Operations and Implementation
- Double Linked List: Operations and Implementations
- Circular Linked List: Concepts and Implementation
- Applications: Stack & Queue Implementation using Linked List
- Polynomial Manipulation using Linked List

1. Introduction to Linked List

1.1 What is a Linked List?

□ **Definition:** A linked list is a linear data structure in which elements (nodes) are NOT stored at contiguous memory locations. Nodes are connected using pointers.

- Each node contains two parts: DATA (value) and LINK (pointer to next node).
- The first node is called the HEAD node — used to traverse the entire list.
- Nodes can be stored at any random memory location and are linked using pointers.
- Such structures which contain a member field pointing to the same structure type are called self-referential structures.

1.2 Why Linked List? (Array Limitations)

Array Limitation	Linked List Advantage
Fixed Size — size must be decided in advance; cannot grow or shrink.	Dynamic Size — nodes can be added or removed at runtime; no memory wasted.
Slow Insertion/Deletion — requires shifting of all elements when inserting or deleting in the middle.	Fast Insertion/Deletion — only pointers are updated; no shifting required.

Array Limitation	Linked List Advantage
Contiguous Memory — requires a single continuous block of memory; fails on fragmented memory.	Non-Contiguous Memory — nodes can be at any memory location, connected via pointers.

1.3 Structure of a Node

□ **Definition:** A node is the basic building block of a linked list. It contains a DATA part (stores the value) and an ADDRESS part (pointer to the next node).

In C, a node is defined using a structure:

```
struct node {
    int data;           // data part
    struct node *next;  // pointer to next node
};
```

□ **Key Point:** The pointer variable 'next' holds the address of the successor node, or NULL (0) if it is the last node.

1.4 Types of Linked Lists

Type	Description
Singly Linked List	Each node has one pointer (next). Traversal is one-directional (forward only). Last node points to NULL.
Doubly Linked List	Each node has two pointers (prev + next). Traversal is bi-directional. First node's prev = NULL, last node's next = NULL.
Circular Linked List	Last node points back to the first node (head) instead of NULL, forming a closed loop.
Circular Doubly Linked List	Combination of Doubly + Circular. Each node has both prev and next pointers, and the list forms a circle.

2. Singly Linked List

2.1 Definition & Structure

□ **Definition:** A singly linked list consists of nodes where each node has a data field and a reference (next) to the next node. The next pointer of the last node is NULL.

Visual Structure: HEAD → [A|→] → [B|→] → [C|→] → [D|NULL]

2.2 Operations on Singly Linked List

Four Main Operations

- Insertion — Add a new node to the linked list
- Deletion — Remove an existing node from the list
- Search — Find a node in the linked list
- Traversal — Access each node of the linked list

2.3 Insertion in Singly Linked List

There are 5 cases for inserting a node in a singly linked list:

Case 1: Insertion at the Beginning

- Allocate space for new node and store data into the data part.
- Make the NEXT pointer of new node point to the existing HEAD (first node).
- Make the new node the new HEAD of the list.

```
newnode = malloc(sizeof(struct node));
if (newnode == NULL) { printf("Memory allocation failed"); return 1; }
scanf("%d", &item);
newnode->data = item;
newnode->next = head;    // new node points to old head
head = newnode;         // new node becomes new head
```

□ **Remember:** ptr->next = head; head = ptr; — Just two pointer updates, no shifting needed!

Case 2: Insertion at the End

- Create new node with given data and set new_node->next = NULL.
- If head == NULL (empty list): make new node the head.
- If head != NULL: traverse to find the last node, then set last_node->next = new_node.

```
newNode = (struct Node *)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
if (head == NULL) { head = newNode; }
else {
    temp = head;
    while (temp->next != NULL) { temp = temp->next; } // traverse to last
    temp->next = newNode; // link new node at end
}
```

Case 3: Insertion at a Specific Position

- Create new node using malloc. If memory fails, display error and stop.
- If position == 1: set newNode->next = head and update head = newNode.

- Otherwise, traverse to the (position-1)th node using temp pointer.
- If temp becomes NULL, display 'Invalid position' and stop.
- Set newNode->next = temp->next and temp->next = newNode.

```
for (i = 1; i < pos - 1 && temp != NULL; i++) { temp = temp->next; }
if (temp == NULL) { printf("Invalid position\n"); return; }
newNode->next = temp->next;
temp->next = newNode;
```

Case 4: Insertion After a Given Node

- Traverse the list from HEAD to find the given node (key).
- If key not found, print 'Node not found' and stop.
- If found: set new_node->next = current->next and current->next = new_node.

```
while (temp != NULL) {
    if (temp->data == key) break;
    temp = temp->next;
}
if (temp == NULL) { printf("Node not found\n"); return; }
newNode->next = temp->next;
temp->next = newNode;
```

Case 5: Insertion Before a Given Node

- If list is empty (head == NULL): print 'List is empty'.
- If first node has the key (head->data == key): set newNode->next = head; head = newNode.
- Otherwise, use two pointers: prev = NULL, temp = head. Traverse until temp->data == key.
- Adjust links: prev->next = newNode and newNode->next = temp.

```
if (head->data == key) { newNode->next = head; head = newNode; }
else {
    prev = head; temp = head->next;
    while (temp != NULL && temp->data != key) { prev = temp; temp = temp->next; }
    if (temp == NULL) { printf("Key not found\n"); return; }
    newNode->next = temp;
    prev->next = newNode;
}
```

2.4 Traversal of Singly Linked List

Traversal means visiting every node of the list from HEAD to the last node.

- Initialize a pointer 'temp' pointing to HEAD of the list.
- Iterate until 'temp' becomes NULL.

- At each step: print temp->data, then move temp = temp->next.

```
struct node *temp = head;
while (temp != NULL) {
    printf("%d --->", temp->data);
    temp = temp->next;
}
```

❑ **Key Point:** Output example for list [12→20→28]: 12--->20--->28--->NULL

2.5 Deletion in Singly Linked List

Case 1: Deletion at Beginning

- If head == NULL: print 'List is empty' and stop.
- Store address of first node in temp = head.
- Move head pointer to next node: head = head->next.
- Free memory of deleted node: free(temp).

```
void deleteFirst() {
    if (head == NULL) { printf("List is empty\n"); return; }
    struct Node *temp = head;
    head = head->next;    // move head to next node
    free(temp);          // free old head
    temp = NULL;
}
```

Case 2: Deletion at End

- Case A — Only ONE node (head->next == NULL): set head = NULL and free the node.
- Case B — Multiple nodes: traverse using two pointers (prev, current) until current->next == NULL. Set prev->next = NULL and free current.

```
while (current->next != NULL) { prev = current; current = current->next; }
if (prev == NULL) { free(head); head = NULL; } // only one node
else { prev->next = NULL; free(current); }
```

Case 3: Deletion at Specific Position

- If head == NULL: print 'List is empty'.
- If position == 1: move head = head->next and free old head.
- Traverse to (position-1)th node using temp.
- Store: del = temp->next; adjust: temp->next = del->next; free(del).

```
struct Node* nodeToDelete = temp->next;
```

```
temp->next = nodeToDelete->next;
free (nodeToDelete);
```

3. Doubly Linked List

3.1 Definition & Structure

□ **Definition:** A Doubly Linked List (two-way linked list) is a linked list where each node contains THREE fields: PREV pointer (address of previous node), DATA (value), and NEXT pointer (address of next node).

Visual Structure: $\text{NULL} \leftarrow [\text{prev}|\text{A}|\text{next}] \rightleftharpoons [\text{prev}|\text{B}|\text{next}] \rightleftharpoons [\text{prev}|\text{C}|\text{next}] \rightleftharpoons [\text{prev}|\text{D}|\text{next}] \rightarrow \text{NULL}$

- Allows traversal in BOTH directions (forward and backward).
- Easier to insert or delete nodes at any point in the list.
- HEAD node's prev = NULL; LAST node's next = NULL.

```
struct node {
    int data;
    struct node *next;    // pointer to next node
    struct node *prev;    // pointer to previous node
};

// Connecting 3 nodes (1, 2, 3):
one->next = two;    one->prev = NULL;
two->next = three;  two->prev = one;
three->next = NULL; three->prev = two;
```

3.2 Traversal of Doubly Linked List

Doubly linked list supports both forward (using next) and backward (using prev) traversal.

```
// Forward traversal
Node* temp = head;
while (temp != NULL) { printf("%d ", temp->data); temp = temp->next; }

// Backward traversal - first go to the last node
while (temp->next != NULL) { temp = temp->next; }
while (temp != NULL) { printf("%d ", temp->data); temp = temp->prev; }
```

3.3 Insertion in Doubly Linked List

Case 1: Insertion at Beginning

- Allocate space for new node and store data.
- Set ptr->prev = NULL and ptr->next = head.
- If head != NULL: set head->prev = ptr.
- Update head = ptr.

```
newnode->data = item;
if (head == NULL) {
    newnode->next = NULL; newnode->prev = NULL; head = newnode;
} else {
    newnode->prev = NULL;
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}
```

Case 2: Insertion at End

- Allocate space for new node and store data.
- If list is empty: make new node the head node.
- Otherwise: traverse to last node using temp, then set temp->next = ptr and ptr->prev = temp.

```
struct Node *newNode = malloc(sizeof(struct Node));
newNode->data = val; newNode->next = NULL; newNode->prev = NULL;
if (*head == NULL) { *head = newNode; return; }
struct Node *temp = *head;
while (temp->next != NULL) { temp = temp->next; }
temp->next = newNode;
newNode->prev = temp;
```

Case 3: Insertion Between Two Nodes

- Traverse to the node after which insertion is needed.
- Set newNode->next = temp->next and newNode->prev = temp.
- Update temp->next->prev = newNode and temp->next = newNode.

3.4 Deletion in Doubly Linked List

Case 1: Deletion at Beginning

Store first node in ptr. Move head to next node. Set head->prev = NULL. Free ptr.

```
void deleteBeginning() {
    if (head == NULL) { printf("List is empty\n"); return; }
    struct node *temp = head;
    head = head->next;
    if (head != NULL) head->prev = NULL;
}
```

```
    free(temp);  
}
```

Case 2: Deletion at End

- Case A — Only ONE node: `free(head); head = NULL;`
- Case B — Multiple nodes: traverse to last node. Set `temp->prev->next = NULL` and `free(temp)`.

```
if (head->next == NULL) { free(head); head = NULL; return; }  
struct node *temp = head;  
while (temp->next != NULL) { temp = temp->next; }  
temp->prev->next = NULL;    // Disconnect last node  
free(temp);
```

Case 3: Deletion Between Two Nodes

- Traverse to find the node to delete.
- Set `del->prev->next = del->next` and `del->next->prev = del->prev`.
- Free the deleted node.

4. Circular Linked List

4.1 Definition & Structure

□ **Definition:** A Circular Linked List is a data structure where the LAST node points back to the FIRST node (HEAD) instead of NULL, forming a closed loop.

Visual Structure: $\rightarrow [1|next] \rightarrow [2|next] \rightarrow [3|next] \rightarrow (\text{back to } 1)$

- No node contains NULL pointer in the next field (except in empty list).
- Can be traversed starting from any node.
- Both singly and doubly linked lists can be made circular.

```
struct node { int data; struct node *next; };  
  
/* Connect as circular: three->next = one; */
```

4.2 Insertion in Circular Linked List

Case 1: Insertion at Beginning

- Allocate new node and store data.
- Traverse to find the LAST node (temp) where `temp->next == head`.

- Set ptr->next = head (new node points to old head).
- Set temp->next = ptr (last node now points to new node).
- Set head = ptr (new node becomes new head).

```
if (head == NULL) { newNode->next = newNode; head = newNode; }
else {
    struct Node *temp = head;
    while (temp->next != head) { temp = temp->next; } // find last node
    newNode->next = head;
    temp->next = newNode;
    head = newNode;
}
```

Case 2: Insertion at End

- If list is empty: set newNode->next = newNode and head = newNode.
- Otherwise: traverse to last node (temp->next == head). Set temp->next = newNode and newNode->next = head.

```
void insertEnd(int value) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = value;
    if (head == NULL) { head = newNode; newNode->next = head; }
    else {
        temp = head;
        while (temp->next != head) { temp = temp->next; }
        temp->next = newNode;
        newNode->next = head;
    }
}
```

4.3 Deletion in Circular Linked List

Case 1: Deletion at Beginning

- Traverse using pointer ptr to reach the LAST node.
- Set ptr->next = head->next (last node now points to second node).
- Free the old head node.
- Set head = ptr->next (second node becomes new head).

```
if (head->next == head) { free(head); head = NULL; return; }
struct node *temp = head, *current = head;
while (current->next != head) { current = current->next; }
current->next = head->next;
head = head->next;
free(temp);
```

Case 2: Deletion at End

- Case A — ONE node ($\text{head} \rightarrow \text{next} == \text{head}$): $\text{free}(\text{head})$; $\text{head} = \text{NULL}$.
- Case B — Multiple nodes: use two pointers (prev , ptr). Traverse until $\text{ptr} \rightarrow \text{next} == \text{head}$. Set $\text{prev} \rightarrow \text{next} = \text{head}$ and $\text{free}(\text{ptr})$.

```
void deleteEnd() {
    if (head == NULL) { printf("List is empty\n"); return; }
    if (head->next == head) { free(head); head = NULL; return; }
    temp = head; prev = NULL;
    while (temp->next != head) { prev = temp; temp = temp->next; }
    prev->next = head; // second-last now points to head
    free(temp);
}
```

4.4 Circular Doubly Linked List

□ **Definition:** A Circular Doubly Linked List is a mixture of doubly linked list and circular linked list. Each node has TWO pointers (prev + next), and the last node's next points to the first node while the first node's prev points to the last node.

- Combines bi-directional traversal (doubly) with no NULL end (circular).
- Useful in applications like music players (prev / next song in loop).

5. Comparison: Types of Linked Lists

Feature	Singly Linked List	Doubly Linked List
Pointers per node	1 (next only)	2 (prev + next)
Traversal direction	Forward only	Forward + Backward
Memory usage	Less (one pointer)	More (two pointers)
Deletion efficiency	Need prev node explicitly	Easy — prev pointer available
Insertion at beginning	$O(1)$	$O(1)$
Insertion at end	$O(n)$ — traverse to last	$O(n)$ — traverse to last
NULL pointers	Last node's next = NULL	Head's prev = NULL, Last's next = NULL
Circular variant	Yes (last $\rightarrow$ head)	Yes (last $\rightarrow$ head, head $\leftarrow$ last)

6. Stack Implementation Using Linked List

6.1 Concept

□ **Definition:** A Stack is a linear data structure following LIFO (Last In First Out). When implemented using a linked list, PUSH adds a node at the beginning and POP removes from the beginning.

- No overflow condition (unlike array) — limited only by heap memory.
- TOP of stack = HEAD of linked list.
- PUSH = Insert at beginning of linked list.
- POP = Delete from beginning of linked list.

□ **Remember:** In linked list stack: `newNode->next = top; top = newNode;` (for PUSH)

6.2 Node Structure for Stack

```
struct node {
    int data;
    struct node *next;
};
struct node *top = NULL;
```

6.3 PUSH Operation (Insert at Beginning)

Algorithm: Allocate new node → store value → set `newNode->next = top` → set `top = newNode`.

```
void push(int value) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) { printf("Stack Overflow\n"); return; }
    newNode->data = value;
    newNode->next = top;    // new node points to current top
    top = newNode;         // new node becomes the top
}
```

6.4 POP Operation (Delete from Beginning)

Algorithm: If `top == NULL` → Underflow. Else, store `top` in `temp`, move `top = top->next`, print `temp->data`, `free(temp)`.

```
void pop() {
    if (top == NULL) { printf("Stack Underflow\n"); return; }
    struct node *temp = top;
    printf("Deleted element: %d\n", temp->data);
    top = top->next;
    free(temp);
}
```

6.5 PEEK & Display Operations

```

void peek() {
    if (top == NULL) printf("Stack is empty\n");
    else printf("Top element: %d\n", top->data);
}

void display() {
    if (top == NULL) { printf("Stack is empty\n"); return; }
    struct node *temp = top;
    while (temp != NULL) { printf("%d\n", temp->data); temp = temp->next; }
}

```

6.6 Example — Stack [10, 20, 30, 40, 50]

Stack (TOP = 50): 50 → 40 → 30 → 20 → 10 → NULL

After PUSH(60): 60 → 50 → 40 → 30 → 20 → 10 → NULL

After POP(): removes 60 → 50 → 40 → 30 → 20 → 10 → NULL

Advantages of Linked List Stack over Array Stack

- No fixed size limit — grows dynamically
- No overflow (unless memory is exhausted)
- Efficient memory usage — only allocated when needed
- No need to pre-define size

7. Queue Implementation Using Linked List

7.1 Concept

□ **Definition:** A Queue is a linear data structure following FIFO (First In First Out). Using linked list: ENQUEUE (insert) at the REAR end, DEQUEUE (delete) from the FRONT end.

- Two pointers maintained: FRONT (head/start) and REAR (tail/end).
- FRONT points to the first element (dequeue from here).
- REAR points to the last element (enqueue here).
- No overflow condition unlike array queue.

Visual: FRONT → [22|→] → [-5|→] → [19|→] → [7|NULL] ← REAR

7.2 ENQUEUE Operation (Insert at Rear)

Algorithm: Allocate new node → store value → set newNode->next = NULL → if queue empty: front = rear = newNode → else: rear->next = newNode; rear = newNode.

```

void enqueue(int value) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) { printf("Queue Overflow\n"); return; }
}

```

```

newNode->data = value;
newNode->next = NULL;
if (front == NULL) { front = rear = newNode; }
else { rear->next = newNode; rear = newNode; }
printf("%d inserted\n", value);
}

```

7.3 DEQUEUE Operation (Delete from Front)

Algorithm: If front == NULL → Underflow. Else store front in temp, print front->data, move front = front->next, if front == NULL then rear = NULL, free(temp).

```

void dequeue() {
    if (front == NULL) { printf("Queue Underflow\n"); return; }
    struct node *temp = front;
    printf("%d deleted\n", front->data);
    front = front->next;
    if (front == NULL) rear = NULL; // queue became empty
    free(temp);
}

```

7.4 Display Queue

```

void display() {
    if (front == NULL) { printf("Queue is empty\n"); return; }
    struct node *temp = front;
    printf("Queue elements:\n");
    while (temp != NULL) { printf("%d ", temp->data); temp = temp->next; }
    printf("\n");
}

```

7.5 Example — Queue using Linked List

Initial Queue: FRONT → [3|→] → [4|→] → [5|→] → [6|→] → [7|→] → [8|NULL] ← REAR

After ENQUEUE(9): FRONT → [3|→] → [4|→] → [5|→] → [6|→] → [7|→] → [8|→] → [9|NULL] ← REAR

After DEQUEUE(): removes 3 → FRONT → [4|→] → [5|→] → [6|→] → [7|→] → [8|→] → [9|NULL] ← REAR

Advantages of Linked List Queue over Array Queue

- No fixed size — grows and shrinks dynamically
- No wasted memory (unlike circular queue complexity)
- No overflow unless memory exhausted
- Simple and clean implementation

8. Polynomial Manipulation Using Linked List

8.1 What is Polynomial Manipulation?

□ **Definition:** A polynomial is an expression containing a sum of powers in one or more variables multiplied by coefficients. Example: $5x^2 + 2x - 3$

- Linked lists are ideal for representing polynomials since terms may have non-consecutive powers.
- Each NODE of the linked list represents ONE TERM of the polynomial.

8.2 Node Structure for Polynomial

Each node has THREE fields:

Field	Purpose
Coefficient	The numeric value of the term (e.g., 5 in $5x^2$)
Degree (Power)	The exponent/power of the variable x (e.g., 2 in $5x^2$)
Address (next)	Pointer to the next term node in the linked list

```
struct poly {
    int coeff;           // coefficient
    int degree;          // power of x
    struct poly *next;   // pointer to next term
};
```

8.3 Example — Polynomial Representation

Polynomial: $4x^3 + 6x^2 + 10x + 6$

Linked List: $\text{poly} \rightarrow [4|3| \rightarrow] \rightarrow [6|2| \rightarrow] \rightarrow [10|1| \rightarrow] \rightarrow [6|0|\text{NULL}]$

Polynomial: $5x^2 + 4x^1 + 2$

Linked List: $\text{poly} \rightarrow [5|2| \rightarrow] \rightarrow [4|1| \rightarrow] \rightarrow [2|0|\text{NULL}]$

8.4 Addition of Two Polynomials

Rules for polynomial addition:

- Scan both polynomials from left to right simultaneously.
- If degrees are EQUAL: add the coefficients and create a new node in result list.
- If degree of P1 > degree of P2: copy term from P1 to result list.
- If degree of P1 < degree of P2: copy term from P2 to result list.
- Continue until both lists are exhausted.

Example 1: Add $5x^2 + 4x^1 + 2$ and $-5x^1 - 5$

List 1: $[5|2] \rightarrow [4|1] \rightarrow [2|0]$

List 2: $[-5|1] \rightarrow [-5|0]$

Step 1: Degree of L1 = 2, L2 = 1 → L1 higher → copy [5|2] to result
 Step 2: Both degree 1 → add: $4 + (-5) = -1$ → add [-1|1] to result
 Step 3: Both degree 0 → add: $2 + (-5) = -3$ → add [-3|0] to result

Result: $5x^2 - x - 3$

Result List: [5|2] → [-1|1] → [-3|0]

Example 2: Add $5x^3 + 4x^2 + 2$ and $5x^1 - 5$

List 1: [5|3] → [4|2] → [2|0]

List 2: [5|1] → [-5|0]

Result: $5x^3 + 4x^2 + 5x - 3$

Result List: [5|3] → [4|2] → [5|1] → [-3|0]

General Example: Add $(5x^4 + 6x^2 + 10x - 6)$ and $(7x^3 + 3x^2 + 2x + 7)$

Result: $5x^4 + 7x^3 + 9x^2 + 12x + 1$

Polynomial Addition Algorithm Summary

- Traverse both polynomial lists simultaneously using two pointers p1 and p2
- Compare degrees: equal → add coefficients; p1 higher → copy p1 term; p2 higher → copy p2 term
- Move the pointer(s) whose term was just processed
- Continue until both lists are exhausted
- Append any remaining terms from either list to the result

9. Quick Revision Summary

9.1 Key Terminologies

Term	Meaning
Head / HEAD	Pointer to the first node of the linked list
Tail / Last Node	The last node; its next = NULL (or HEAD in circular)
NULL	End marker — indicates no further node
Traversal	Visiting each node one by one from HEAD to last
Self-referential structure	Struct containing a pointer to its own type
malloc()	Dynamic memory allocation in C for new nodes
free()	Deallocate/release memory of a deleted node
LIFO	Last In First Out — Stack principle
FIFO	First In First Out — Queue principle

Term	Meaning
PUSH	Insert element at TOP of stack (beginning of linked list)
POP	Delete element from TOP of stack (beginning of linked list)
Enqueue	Insert element at REAR of queue (end of linked list)
Dequeue	Delete element from FRONT of queue (beginning of linked list)
Coefficient	Numeric value of a polynomial term
Degree / Power	Exponent of variable in a polynomial term

9.2 Operations Summary Table

Operation	Singly Linked List	Doubly Linked List
Insert at Beginning	newNode->next = head; head = newNode;	Also set head->prev = newNode; newNode->prev = NULL;
Insert at End	Traverse to last; last->next = newNode;	Also set newNode->prev = last;
Delete at Beginning	head = head->next; free(old head);	Also set head->prev = NULL;
Delete at End	Traverse; prev->next = NULL; free(last);	last->prev->next = NULL; free(last);
Traversal	while(temp!=NULL){...temp=temp->next;}	Forward and backward using prev/next

9.3 Important Practice Questions

Questions to Practice

- Write a C program to insert a node at beginning, end, and specific position in SLL.
- Write a C program to delete a node from beginning, end, and specific position in SLL.
- Explain the difference between Singly, Doubly, and Circular Linked List with diagrams.
- Implement Stack using Linked List with PUSH, POP, and PEEK operations.
- Implement Queue using Linked List with ENQUEUE and DEQUEUE operations.
- Add two polynomials $(5x^3 + 4x^2 + 2)$ and $(5x - 5)$ using linked list.
- What are the advantages of linked list over array? Explain with examples.
- Write a C function to traverse and display all elements of a circular linked list.
- How is deletion at beginning different in Singly vs Doubly Linked List?
- Add polynomials: $(3x^4 + 2x^2 + 5)$ and $(4x^3 + x^2 + 3x - 2)$.

10. References & Resources

10.1 Text Books

- "Data Structure: A Pseudo code approach with C" — Richard F. Gilberg & Behrouz A. Forouzan, Thomson Publication, 2nd Ed.
- "Data Structure in C" — A.M. Tanenbaum, Y. Langsam, M.J. Augenstein, PHI/Pearson, 7th Ed.

10.2 Reference Books

- "Data Structures with C" (Schaum's Series) — Seymour Lipschutz, Tata-McGraw-Hill, 1st Ed.
- "Fundamentals of Data Structure in C" — Horowitz, Sahani & Freed, Computer Science Press, 2nd Ed.
- "Data Structures Using C" — Reema Thareja, Oxford University Press, 2nd Ed.

10.3 Online Sources

- <https://www.javatpoint.com/data-structure-introduction>
- <https://www.geeksforgeeks.org/data-structures/linked-list/>
- <https://www.geeksforgeeks.org/doubly-linked-list/>
- <https://www.geeksforgeeks.org/circular-linked-list/>
- <https://btechx.infinityfreeapp.com>

— End of Unit II Notes —

BTechX